

Informe de Auditoría del Sistema Web

Integrantes del grupo:

Jorge Illera Rivera

Iker Argulo Galán

Oscar Lijerón Pérez

Bruno Izaguirre Martínez de Marañón

María Bogajo

Ander Javier Corral

Fecha:

09/11/2024

Índice de Contenidos

1	Introducción	3
2	Metodología	3
3	Vulnerabilidades	3
3.1	Rotura de control de acceso	3
3.1.1	Soluciones implementadas	3
3.2	Fallos criptográficos	3
3.2.1	Soluciones implementadas	3
3.3	Inyección	3
3.3.1	Cross-Site Scripting (XSS)	3
3.3.2	Inyección SQL	5
3.3.3	Soluciones implementadas	5
3.4	Diseño inseguro	6
3.4.1	Soluciones implementadas	6
3.5	Configuración de seguridad insuficiente	6
3.5.1	Soluciones implementadas	6
3.6	Componentes vulnerables y obsoletos	6
3.6.1	Soluciones implementadas	6
3.7	Fallos de identificación y autenticación	6
3.7.1	Soluciones implementadas	6
3.8	Fallos en la integridad de datos y software	6
3.8.1	Soluciones implementadas	6
3.9	Fallos en la monitorización de la seguridad	6
3.9.1	Soluciones implementadas	6
4	Conclusiones	6
5	Bibliografía	6
6	Apéndices	6

Índice de Figuras

1	Errores de inyección detectados por ZAP.	4
---	--	---

Índice de Tablas

1 Introducción

Este informe detalla la auditoría de seguridad del sistema web utilizando OWASP ZAP, con el objetivo de identificar y mitigar vulnerabilidades.

2 Metodología

La auditoría se realizó mediante un análisis de ZAP, ejecutando...

3 Vulnerabilidades

3.1 Rotura de control de acceso

3.1.1 Soluciones implementadas

3.2 Fallos criptográficos

3.2.1 Soluciones implementadas

3.3 Inyección

En el apartado de inyección se han detectado dos amenazas de riesgo alto. Las vulnerabilidades encontradas son Cross-Site Scripting e Inyección SQL, como se muestra en la Figura 1.

3.3.1 Cross-Site Scripting (XSS)

La vulnerabilidad CWE-79 corresponde a Cross-Site Scripting (XSS) Reflejado, que permite a un atacante inyectar código malicioso en los parámetros de entrada del sistema, como los campos de formularios o las URL. Cuando un usuario accede a una URL o un formulario que contiene este código inyectado, el navegador del usuario ejecuta dicho código, lo que puede llevar a la manipulación de datos, el robo de cookies, la ejecución de scripts no autorizados, y otros ataques de seguridad.

En este caso, la vulnerabilidad fue detectada en campos de entrada como la fecha de lanzamiento y otros campos en el formulario de modificación de los juegos.

Causa de la Vulnerabilidad

La vulnerabilidad XSS detectada se originaba en el campo de la fecha de lanzamiento de los juegos y otros campos de entrada en el formulario de modificación. Esto se debía a que los valores de entrada eran reflejados en la respuesta sin la sanitización o validación adecuada, permitiendo a los atacantes inyectar y ejecutar código JavaScript en el navegador del usuario.

Pruebas y Comportamiento Observado

Sin la sanitización de entrada: OWASP ZAP detectó que ciertos campos permitían la inyección de código JavaScript en los valores de los formularios. Por ejemplo, en el campo de la fecha, se podía ingresar un valor como `<script>alert(1);</script>`, lo cual provocaba la ejecución de un alert en el navegador.

Después de implementar soluciones: Tras aplicar la sanitización y validación, el escaneo de ZAP no encontró vulnerabilidades de XSS, lo que confirmó que el problema estaba resuelto.



Figura 1: Errores de inyección detectados por ZAP.

Soluciones Aplicadas

Para mitigar la vulnerabilidad de XSS y otros posibles ataques de inyección, se implementaron las siguientes soluciones:

- **Sanitización de Entrada y Escapado de Salida:**

Se utilizó la función `htmlspecialchars()` en PHP para escapar caracteres especiales (como `<`, `>`, `'`, `"`, `&`) en todas las salidas. Esto asegura que el contenido reflejado en el navegador sea tratado como texto y no como código ejecutable.

- **Validación y Sanitización de Fecha de Lanzamiento:**

Se implementó una expresión regular (`preg_match`) para asegurarse de que las fechas ingresadas cumplan con el formato `YYYY-MM-DD`.

- **Sanitización de Datos Entrantes con `mysqli_real_escape_string`:**

Para cada valor de entrada recibido a través del formulario, se aplicó `mysqli_real_escape_string` para evitar que cualquier código especial o malicioso llegue a la base de datos.

- **Validación de Campos Numéricos (`rating` y `price`):**

Los valores de `rating` y `price` se convirtieron a tipo `float` usando `floatval()` para asegurarse de que se almacenan y procesan correctamente como valores numéricos.

Resultados de la Segunda Auditoría

Después de implementar las soluciones descritas, se realizó un segundo análisis con OWASP ZAP. Los resultados fueron positivos: no se detectaron vulnerabilidades de XSS ni otros problemas de inyección.

Conclusiones

La vulnerabilidad de XSS reflejada fue una amenaza crítica para la aplicación, ya que podía permitir a los atacantes ejecutar código malicioso en el navegador de los usuarios. Gracias a la implementación de sanitización y validación estrictas en los campos de entrada y salida, esta vulnerabilidad se ha mitigado de manera efectiva. La aplicación ahora cumple con un estándar de seguridad más alto, protegiendo los datos y la experiencia de los usuarios frente a posibles ataques de inyección de código.

La vulnerabilidad **CWE-89: Inyección SQL** también se ha solucionado con estos cambios.

3.3.2 Inyección SQL

Descripción: La vulnerabilidad CWE-89 corresponde a la inyección SQL, donde un atacante es capaz de insertar o manipular consultas SQL a través de los datos de entrada, en este caso, el parámetro `id` en la URL. En el error encontrado, la inyección fue posible al modificar el parámetro de la siguiente manera:

```
1750' AND '1'='1' --
```

Esto permitió al atacante modificar la consulta SQL ejecutada en el servidor, manipulando los resultados devueltos y accediendo a información que no debía ser revelada. En particular, se manipuló la consulta para obtener datos utilizando una condición booleana que siempre era verdadera, `'1'='1'`, y luego se comentó el resto de la consulta utilizando `--`, lo que invalidaba el resto de la consulta SQL.

Causa de la Vulnerabilidad: La vulnerabilidad fue causada por la construcción dinámica de consultas SQL sin la adecuada sanitización ni parametrización de los datos proporcionados por el usuario. En este caso, el valor del parámetro `id` se concatenó directamente a la consulta SQL sin protección, lo que permitió la ejecución de código SQL malicioso.

3.3.3 Soluciones implementadas

- Utilizar consultas SQL parametrizadas para evitar la inyección de código malicioso.
- Implementar la sanitización de todas las entradas del usuario.
- No confiar en la validación del lado del cliente; la verificación debe realizarse siempre en el servidor.
- Aplicar el principio de privilegio mínimo en el acceso a la base de datos, utilizando usuarios con permisos limitados.
- Utilizar procedimientos almacenados siempre que sea posible para encapsular las consultas SQL.

3.4 Diseño inseguro

3.4.1 Soluciones implementadas

3.5 Configuración de seguridad insuficiente

3.5.1 Soluciones implementadas

3.6 Componentes vulnerables y obsoletos

3.6.1 Soluciones implementadas

3.7 Fallos de identificación y autenticación

3.7.1 Soluciones implementadas

3.8 Fallos en la integridad de datos y software

3.8.1 Soluciones implementadas

3.9 Fallos en la monitorización de la seguridad

3.9.1 Soluciones implementadas

4 Conclusiones

5 Bibliografía

References

- [1] OWASP, "Cross-Site Scripting (XSS)", <https://owasp.org/www-community/attacks/xss/>,
accedido el 3 de noviembre de 2024.

6 Apéndices